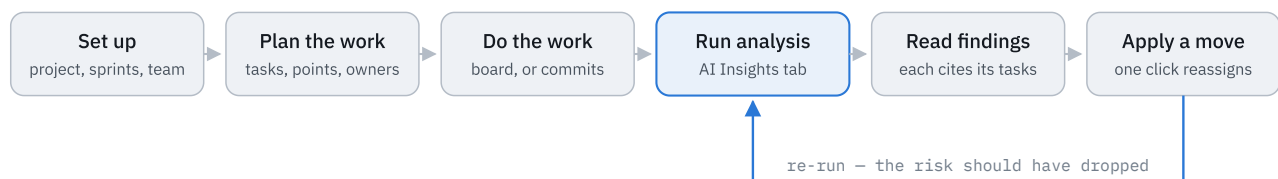


Three flows through TeamSync

What a manager does, what one analysis actually does across the two services, and what happens when work lands in GitHub. Every number in the product is computed from a knowledge graph of the project, and every warning carries the tasks and people it came from — these three pictures are where that shows up.

FLOW 1

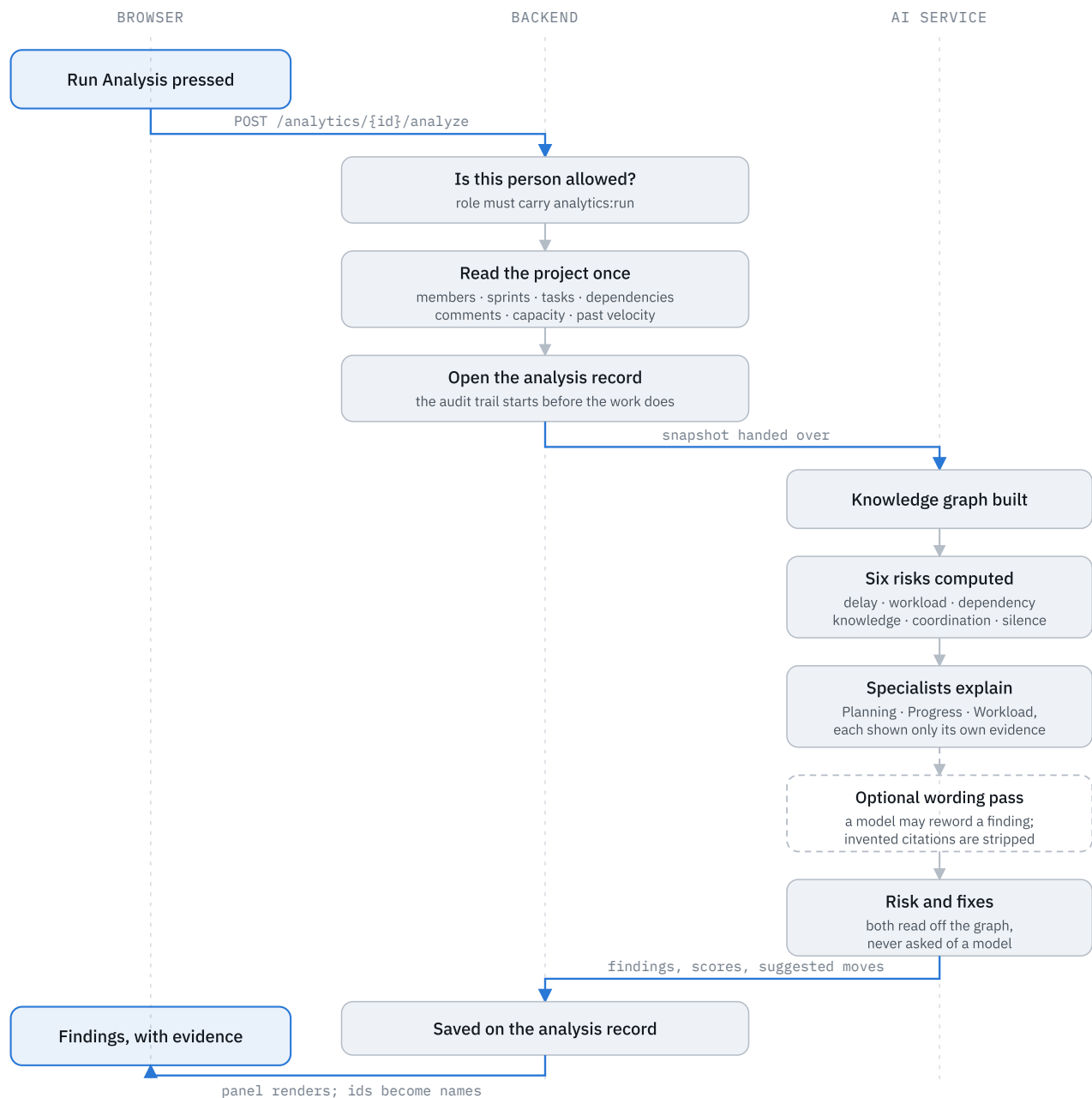
The loop a manager runs



Capacity, sprint dates and task components are set once; they are what the pace, workload and knowledge signals read.

The product's claim is testable by the person using it: apply what it suggests, analyse again, and the risk it predicted should fall. On 1,524 real sprints the predicted effect held every time.

One analysis, hop by hop

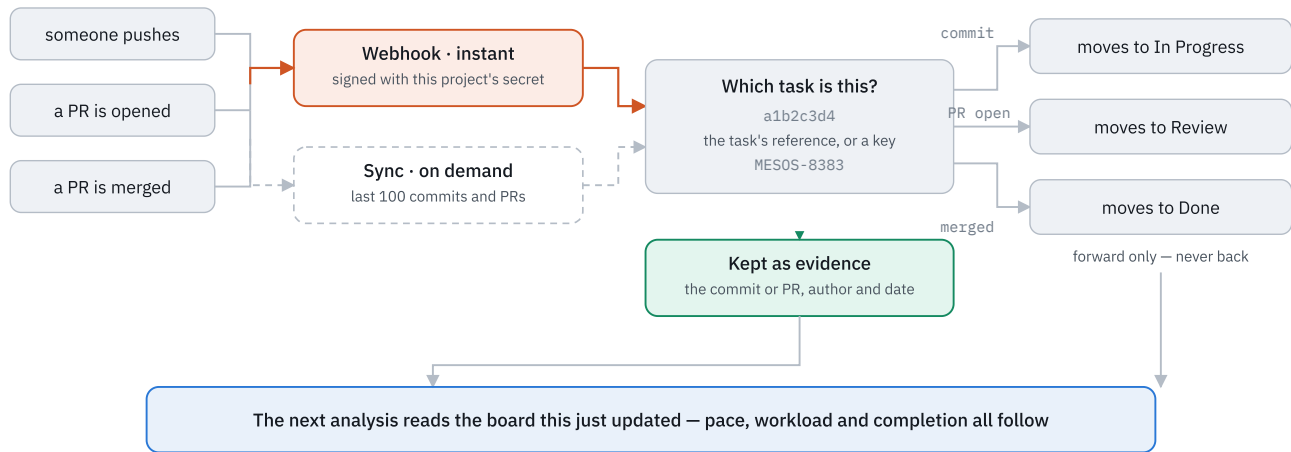


The project is read once per analysis and handed over as a snapshot — the AI service holds no database of its own. Typical timing is about **700 ms** end to end, of which the agent pipeline is under 50 ms; the rest is reading the project and saving the result. Without an API key the optional wording pass is skipped and the deterministic text stands, which is how every published result was measured.

FLOW 3

Work arriving from GitHub

IN THE REPOSITORY



Both routes run the same matching engine, so a commit behaves identically whether GitHub pushed it or someone pressed Sync. A push also knows its branch, which the repository listing does not, so `fix/a1b2c3d4-retry` moves that task even when the commit message says nothing. Every webhook call must carry a signature made with the project's own secret; one that doesn't is refused before its body is read.

The rule underneath all three

Nothing in this chain asks a language model whether a risk exists. Risks are computed from the graph, each finding carries the node ids that witness it, and those ids are checked against the graph before anyone sees them — 40,001 of 40,001 resolved across 2,961 analyses of real sprints. That is also why a status change driven by GitHub keeps the commit that caused it.